

MY SERVICE PARKING

Arquitetura e Plano de Desenvolvimento

Guia de organização para desenvolvimento em dupla

Projeto	MY Service Parking
Equipe	Marcos Vinicius da Silva Zacchi Yan Carlos Farias
Arquitetura	Front-end Vue.js + API Gateway + Microserviços
Versão do documento	1.0 - 19/08/2026

Base documental

Este plano foi organizado a partir de “Estacionamento - MY Service Parking.docx” (visão e fluxo operacional) e “Atributo Prioritário.docx” (requisitos funcionais, não funcionais e atributos de qualidade).

1. Objetivo deste documento

Este documento transforma os requisitos do MY Service Parking em um plano de implementação compartilhado. Ele deve servir como referência para decisões de arquitetura, divisão de responsabilidades, contratos de API, estrutura do front-end, fluxo de Git e acompanhamento das entregas.

- Evitar que os dois desenvolvedores implementem a mesma funcionalidade de formas diferentes.
- Definir limites claros entre front-end, API Gateway e microserviços.
- Permitir desenvolvimento paralelo com contratos de API combinados antes da implementação.
- Manter rastreabilidade entre requisitos, serviços e telas.
- Separar o MVP necessário para funcionar da automação com sensores e dispositivos.

2. Visão do produto e escopo

O MY Service Parking é um sistema de controle de estacionamento voltado a motoristas rotativos e mensalistas. No fluxo de visão do produto, a operação é assistida por câmera e atendente: o atendente visualiza a placa, informa a placa no sistema, registra a entrada/saída, o sistema calcula a permanência e verifica a situação financeira de mensalistas.

2.1 Decisão de escopo em duas fases

Fase	Objetivo	Inclui
Fase 1 - MVP assistido	Entregar o fluxo principal utilizável sem depender de hardware complexo.	Login; entrada e saída por placa; rotativo e mensalista; cálculo de tarifa/carência; pagamento; cadastro de mensalistas; vagas; relatórios básicos; auditoria/logs essenciais.
Fase 2 - Automação / IoT	Atender requisitos de automação e integração física.	Sensores de vagas; leitura automática/ticket; painéis de LED; sinalização; abertura automática das cancelas; sincronização entre sensores, registros e pagamentos.

Regra de organização

Não bloquear o MVP por causa de sensores/cancelas. Criar interfaces de integração desde o início e implementar o hardware real na Fase 2.

3. Arquitetura escolhida

A arquitetura adotada para o projeto é uma aplicação web com front-end em Vue.js, comunicação HTTP/REST por um API Gateway e back-end dividido em microserviços por responsabilidade de negócio. Cada serviço deve ser independente o suficiente para evoluir sem exigir alterações em todo o sistema.

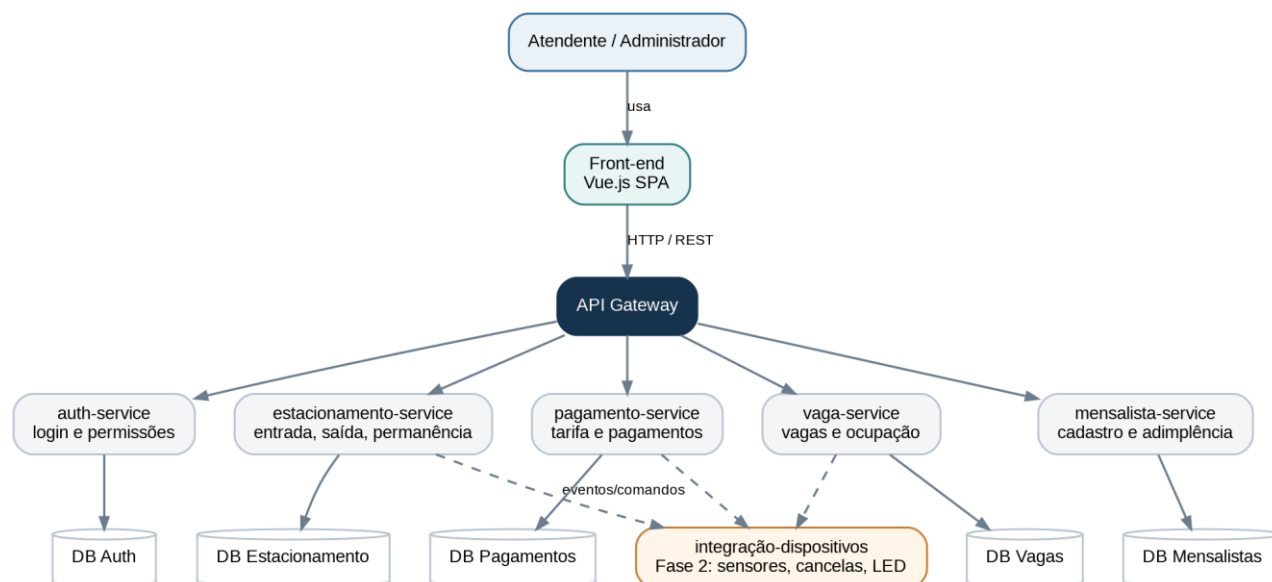


Figura 1 - Arquitetura lógica proposta para o MY Service Parking.

3.1 Princípios da arquitetura

- Front-end único em Vue.js (SPA) para atendentes e administradores.
- API Gateway como ponto único de entrada do front-end para o back-end.
- Microserviços separados por domínio: autenticação, estacionamento, mensalistas, pagamentos e vagas.
- Banco por serviço como direção arquitetural; evitar tabelas compartilhadas diretamente entre serviços.
- Comunicação inicial síncrona via REST; eventos assíncronos podem ser adicionados depois se houver necessidade.
- Integração com hardware encapsulada em uma camada/serviço de dispositivos, evitando espalhar código de sensor/cancela pelos serviços de negócio.

3.2 Tecnologias de referência

Camada	Tecnologia sugerida	Responsabilidade
Front-end	Vue.js + Vue Router + Axios	Views, componentes, navegação e consumo das APIs.
Gateway	Node.js + Express	Roteamento, autenticação central, CORS e encaminhamento para serviços.
Microserviços	Node.js + Express	Regras de negócio e APIs REST de cada domínio.
Banco de dados	SQL (ex.: PostgreSQL/MySQL)	Persistência por serviço e integridade dos dados.
Ambiente	Docker / Docker Compose	Subir gateway, serviços e bancos de forma padronizada.
Versionamento	Git + repositório remoto	Branches, pull requests, revisão e histórico.

Observação: as tecnologias de banco, Docker e detalhes de infraestrutura são decisões de implementação propostas neste documento; os anexos de requisitos não determinam uma stack específica.

4. Microserviços e responsabilidades

Serviço	Responsabilidade	Dados principais	Requisitos relacionados
auth-service	Usuários, login, senha, perfis e autorização.	Usuário, Perfil/Sessão	RNF05, RNF06; apoio ao RF12
estacionamento-service	Entrada, saída, permanência, veículo estacionado e status da movimentação.	Veículo, Movimentação	RF01, RF05, RF06, RF08, RF10
mensalista-service	Cadastro de mensalistas, placa vinculada, vencimento e adimplência.	Mensalista, Veículo do mensalista, Plano	Fluxo de mensalista da visão do produto
pagamento-service	Cálculo/registro do pagamento, forma, status e confirmação.	Pagamento, Tabela de preço	RF06, RF07, RF08
vaga-service	Cadastro, tipo, andar, disponibilidade e ocupação.	Vaga, Andar, Evento de ocupação	RF02, RF03, RF04, RF11
integração-dispositivos (Fase 2)	Adaptadores para sensores, cancelas, painéis e leitura de placa/ticket.	Dispositivo, Evento/Comando	RF01-RF05, RF10, RNF10

4.1 Responsabilidades que não devem se misturar

- O front-end não calcula a tarifa final: ele solicita o cálculo ao back-end.
- O front-end não decide se um mensalista pode entrar: ele exibe o resultado do mensalista-service.
- O pagamento-service confirma pagamento; o estacionamento-service registra o encerramento da permanência.
- O vaga-service controla estado de vaga; o estacionamento-service controla a permanência do veículo.
- Nenhum serviço deve acessar diretamente o banco de outro serviço.

5. Organização do front-end em Vue.js

No front-end, “View” representa uma tela completa da aplicação. Componentes são partes reutilizáveis dentro das Views. A navegação será controlada pelo Vue Router e o consumo do back-end ficará centralizado em módulos de services/api.

View	Objetivo	Principais ações
LoginView.vue	Autenticar operador/administrador.	Informar login e senha; receber sessão/token.
DashboardView.vue	Visão operacional rápida.	Exibir vagas livres/ocupadas, veículos no pátio e últimas movimentações.
EntradaView.vue	Registrar entrada.	Informar placa; identificar rotativo/mensalista; confirmar entrada.
SaidaView.vue	Processar saída.	Localizar entrada; calcular tarifa; registrar pagamento; finalizar saída.
MensalistasView.vue	Gerenciar mensalistas.	Cadastrar, consultar, editar status/placa e verificar adimplência.
VagasView.vue	Visualizar e administrar vagas.	Filtrar por andar/tipo/status; acompanhar ocupação.
RelatoriosView.vue	Relatórios administrativos.	Entradas, saídas, ocupação, faturamento e inconsistências.
UsuariosView.vue	Administração de acesso.	Cadastrar usuários/perfis e controlar permissões.

5.1 Estrutura sugerida do repositório



Figura 2 - Organização sugerida dos repositórios/pastas do projeto.

Padrão recomendado

Manter uma pasta docs/ versionada contendo este documento, diagramas, contratos de API, decisões importantes e instruções para subir o ambiente.

6. Fluxos principais do sistema

6.1 Entrada de veículo

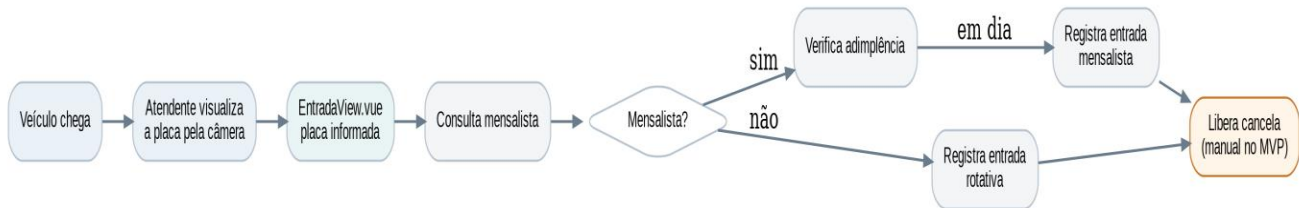


Figura 3 - Fluxo lógico de entrada no MVP.

1. O veículo chega à cancela e a câmera fornece a imagem para o atendente.
2. O atendente informa a placa na EntradaView.vue.
3. O sistema consulta se a placa pertence a um mensalista.
4. Se for mensalista, verifica adimplência; se for rotativo, cria uma permanência rotativa.
5. A entrada é registrada com data/hora exata.
6. No MVP, o atendente libera a cancela; na Fase 2, o comando pode ser automatizado.

6.2 Saída de veículo

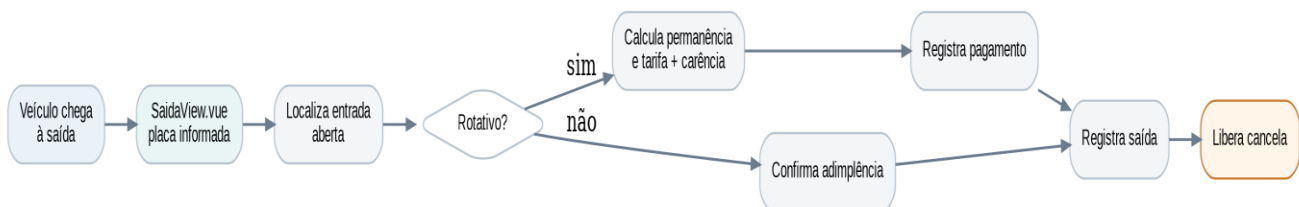


Figura 4 - Fluxo lógico de saída.

7. O atendente informa a placa na SaidaView.vue.
8. O estacionamento-service localiza a movimentação aberta.
9. Para rotativo, o pagamento-service calcula a permanência/tarifa considerando a carência definida no produto.
10. Após o pagamento, o sistema registra a confirmação e encerra a movimentação.
11. Para mensalista, a autorização depende da situação financeira definida no cadastro.
12. A saída é registrada e a cancela é liberada.

7. Modelo de dados inicial

O modelo abaixo é uma proposta de organização por domínio. Os campos podem ser refinados durante a implementação, mas os identificadores e estados principais devem ser combinados antes de os dois devs começarem a codificar as APIs.

Domínio / entidade	Campos mínimos sugeridos
Usuário	id, nome, login/email, senha_hash, perfil, ativo, criado_em
Mensalista	id, nome, documento, status_financeiro, vencimento, ativo
Veículo mensalista	id, mensalista_id, placa, vaga_preferencial/opcional
Movimentação	id, placa, tipo_cliente, entrada_em, saida_em, status, vaga_id/opcional
Pagamento	id, movimentacao_id, valor, forma, status, pago_em
Tabela de preço	id, descrição, valor/hora ou regra, carência_minutos, vigência
Vaga	id, código/número, andar, tipo, status
Auditoria/Log	id, usuario_id, operação, recurso, referência, data_hora

7.1 Estados que devem ser padronizados

Objeto	Estados sugeridos
Movimentação	ABERTA AGUARDANDO_PAGAMENTO FINALIZADA CANCELADA
Pagamento	PENDENTE CONFIRMADO CANCELADO
Mensalista	EM_DIA INADIMPLENTE BLOQUEADO INATIVO
Vaga	LIVRE OCUPADA INDISPONIVEL
Tipo de vaga	COMUM PCD IDOSO GESTANTE ELETRICO

8. Contratos de API iniciais

Antes de implementar front e back em paralelo, combinar payloads e respostas. Os endpoints abaixo são uma proposta inicial; podem mudar, mas a alteração deve ser registrada e comunicada aos dois devs.

Método	Endpoint	Responsável	Uso
POST	/api/auth/login	auth	Autenticar usuário.
POST	/api/entradas	estacionamento	Registrar entrada de veículo.
GET	/api/movimentacoes/aberta?placa={placa}	estacionamento	Localizar permanência aberta.
POST	/api/saidas	estacionamento	Finalizar saída após regras necessárias.
GET	/api/mensalistas/placa/{placa}	mensalista	Consultar mensalista/adimplência.
POST	/api/mensalistas	mensalista	Cadastrar mensalista.
PATCH	/api/mensalistas/{id}	mensalista	Atualizar cadastro/status.
POST	/api/pagamentos/calcular	pagamento	Calcular valor de permanência.
POST	/api/pagamentos	pagamento	Registrar/confirmar pagamento.
GET	/api/vagas	vaga	Listar vagas com filtros.
PATCH	/api/vagas/{id}/status	vaga	Atualizar status (manual/MVP ou sensor/Fase 2).
GET	/api/relatorios/resumo	gateway/agregação	Resumo para dashboard/relatório.

8.1 Exemplo de contrato - registrar entrada

Requisição (exemplo):

```
POST /api/entradas
{
  "placa": "ABC1D23",
  "origem": "OPERADOR"
}
```

Resposta mínima sugerida: id da movimentação, placa, tipo (ROTATIVO/MENSALISTA), data/hora de entrada, status e autorização de acesso.

9. Rastreabilidade dos requisitos

A tabela permite verificar onde cada requisito será tratado. Ela não substitui testes de aceitação, mas ajuda a evitar requisitos “esquecidos” durante a divisão de tarefas.

Req.	Resumo	Serviço(s)	Tela
RF01	Registro de entrada por placa/ticket.	estacionamento + integração-dispositivos	EntradaView
RF02	Ocupação/desocupação por sensores.	vaga + integração-dispositivos	VagasView
RF03	Atualizar vagas nos painéis LED.	vaga + integração-dispositivos	Dashboard/Vagas
RF04	Direcionar para vagas disponíveis.	vaga + integração-dispositivos	Entrada/Vagas
RF05	Controlar cancelas.	estacionamento + integração-dispositivos	Entrada/Saída
RF06	Calcular valor da permanência.	pagamento	SaidaView

RF07	Registrar pagamentos.	pagamento	SaidaView
RF08	Liberar saída após pagamento.	pagamento + estacionamento	SaidaView
RF09	Relatórios de entradas/saídas/ocupação/faturamento.	agregação/relatórios	RelatoriosView
RF10	Rotina de sincronização/inconsistências.	integração + serviços de domínio	Relatorios/Admin
RF11	Vagas especiais.	vaga	VagasView
RF12	Auditoria de operações.	auth + logs/auditoria	Admin/Relatórios

9.1 Requisitos não funcionais que impactam arquitetura

Requisito	Meta do documento-base	Implicação no desenvolvimento
Disponibilidade	99,9% durante 10h-23h.	Health checks, tratamento de falhas, serviços reiniciáveis e infraestrutura estável.
Desempenho - vagas	Atualização até 2 s após sensor.	Processamento eficiente e integração de eventos na Fase 2.
Desempenho - cancela	Abertura até 5 s após confirmação.	Evitar cadeias longas de chamadas no fluxo crítico.
Confiabilidade	Registrar 100% das entradas, saídas, ocupação e pagamentos.	Transações locais, idempotência, logs e reconciliação.
Segurança	Acesso administrativo autenticado e dados protegidos.	Hash de senha, autorização por perfil, validação de entrada e segredos fora do código.
Escalabilidade	Suportar novos andares/sensores.	Evitar regras fixas por andar; modelar vagas/andares por dados.
Usabilidade	Operações principais em até três cliques.	Fluxos curtos e telas focadas por tarefa.
Manutenibilidade	Logs de erros/eventos.	Logger padronizado e correlação de requisições.
Compatibilidade	Integrar sensores, cancelas, LED e pagamento.	Adapters/contratos para dispositivos, isolados do domínio.

10. Organização do desenvolvimento em dupla

A melhor forma de trabalhar em paralelo é dividir por entregas verticais, não apenas “um faz front e outro faz back”. Assim, os dois conhecem o sistema e cada funcionalidade pode ser integrada e testada de ponta a ponta.

10.1 Divisão sugerida

Trilha	Dev Marcos	Dev Yan	Revisão
Fundação	Estrutura Vue, Router, layout base, LoginView.	Gateway, template dos micros serviços, configuração de ambiente.	Ambos revisam contratos e README.
Entrada + mensalista	EntradaView + integração com API; mensalistas no front.	estacionamento-service (entrada) + mensalista-service.	Dev B revisa front; Dev A revisa APIs.
Saída + pagamento	SaidaView + telas de confirmação.	pagamento-service + regras de saída no estacionamento-service.	Teste conjunto de ponta a ponta.
Vagas + dashboard	VagasView/Dashboard e filtros.	vaga-service e endpoints de disponibilidade.	Teste com dados simulados.
Relatórios + qualidade	RelatoriosView e experiência de uso.	Agregação de dados, logs/auditoria e hardening.	Ambos validam requisitos.

Fase 2 IoT	UI de status de dispositivos.	Adapters de sensor/cancela/LED e sincronismo.	Integração em dupla.
------------	-------------------------------	---	----------------------

10.2 Fluxo Git recomendado

- main: somente código estável/integrado.
- develop: integração da versão em desenvolvimento (opcional; usar se a equipe preferir).
- feature/<nome>: uma branch por funcionalidade, exemplo feature/entrada-veiculo.
- fix/<nome>: correções isoladas.
- Pull Request obrigatório para juntar feature em main/develop; o outro dev revisa antes do merge.
- Commits pequenos e descritivos, evitando misturar várias funcionalidades no mesmo commit.

Regra essencial

Antes de alterar um endpoint ou payload que o outro dev já está usando, atualizar o contrato em docs/ e avisar a equipe. Isso reduz quebras entre front-end e microserviços.

11. Roadmap sugerido de implementação

Etapa	Entrega	Critério de conclusão
0 - Preparação	Repositórios, README, ambiente, padrão de branches, contratos iniciais.	Ambos conseguem clonar e subir os projetos.
1 - Autenticação	Login funcional + proteção de rotas.	Usuário autorizado entra; não autorizado é bloqueado.
2 - Entrada	Registrar placa e criar movimentação; consultar mensalista.	Entrada rotativa e mensalista persistidas corretamente.
3 - Saída/Pagamento	Calcular permanência, aplicar regra de tarifa/carência, pagar e finalizar.	Fluxo completo de rotativo concluído.
4 - Mensalistas	CRUD e status financeiro.	Consulta por placa responde estado corretamente.
5 - Vagas	Cadastro/estado/tipos especiais.	Dashboard e VagasView refletem disponibilidade.
6 - Relatórios/Auditoria	Entradas, saídas, ocupação, faturamento e logs.	Admin consulta dados e ações relevantes ficam registradas.
7 - Qualidade	Testes, erros, segurança básica, documentação e Docker.	Ambiente reproduzível e requisitos críticos validados.
8 - Automação IoT	Sensores, painéis, cancelas e sincronização.	Requisitos físicos integrados e medidos.

12. Definition of Done (DoD)

Uma funcionalidade só deve ser considerada concluída quando todos os itens aplicáveis abaixo estiverem atendidos:

- Código executa sem erros no ambiente local dos dois devs.
- Endpoint e payload estão documentados quando houver API.
- Validações básicas e mensagens de erro foram tratadas.
- Front-end não contém regra de negócio que deveria estar no serviço.
- Dados persistidos foram testados com casos válidos e inválidos.

- Permissões/autenticação foram consideradas quando a função é administrativa.
- Logs relevantes existem no back-end.
- Pull Request foi revisado pelo outro dev.
- README/documentação foi atualizado se a forma de executar mudou.
- Requisito relacionado foi marcado como coberto/testado.

13. Quadro de acompanhamento

Item	Responsável	Status	Dependência / observação
Definir banco e ORM	A definir	Pendente	Decidir antes do modelo físico.
Criar front Vue + Router	A definir	Pendente	Base para todas as Views.
Criar API Gateway	A definir	Pendente	Rotas e padrão de erro.
Criar auth-service	A definir	Pendente	Necessário para áreas administrativas.
Criar estacionamento-service	A definir	Pendente	Núcleo de entrada/saída.
Criar mensalista-service	A definir	Pendente	Consulta por placa.
Criar pagamento-service	A definir	Pendente	Definir regra exata de preços.
Criar vaga-service	A definir	Pendente	Modelar andares e tipos especiais.
Construir Dashboard/Views	A definir	Pendente	Usar APIs ou mocks temporários.
Relatórios/auditoria	A definir	Pendente	Definir recortes e filtros.
Docker Compose	A definir	Pendente	Subir todos os serviços.
Integração IoT	Fase 2	Pendente	Depende de definição dos dispositivos/protocolos.

14. Decisões ainda abertas

Os documentos-base definem o comportamento esperado, mas não especificam todos os detalhes técnicos. A equipe deve fechar estas decisões antes ou durante as primeiras etapas:

- Qual banco SQL será utilizado (PostgreSQL, MySQL ou outro).
- Qual ORM/biblioteca de acesso a dados será adotado.
- Regra completa da tabela de preços: valor da primeira hora, horas adicionais, arredondamento e carência de 10 minutos.
- Como mensalidades e vencimentos serão atualizados no projeto acadêmico (manual, simulado ou integração externa).
- Como a câmera será representada no MVP: vídeo real, stream local ou apenas espaço/placeholder de demonstração.
- Quais sensores/cancelas/painéis serão simulados e quais terão hardware real na Fase 2.
- Qual formato de autenticação será usado (sessão ou JWT).
- Se relatórios permanecerão como agregação via gateway ou ganharão um reporting-service separado.

Prioridade

Fechar primeiro as decisões que bloqueiam contratos: regra de tarifa, banco/ORM, formato de autenticação e payloads de entrada/saída.

15. Referências internas do projeto

- Estacionamento - MY Service Parking.docx - visão do produto e fluxo operacional de entrada, saída e mensalistas.
- Atributo Prioritário.docx - requisitos funcionais RF01-RF12, requisitos não funcionais RNF01-RNF10 e atributos de disponibilidade, desempenho e confiabilidade.
- Este documento - decisões e propostas de arquitetura/organização para orientar a implementação.

Nota de governança: quando uma decisão técnica for alterada, registrar a mudança no repositório (docs/) para manter os dois desenvolvedores alinhados.